

Instruction Level Parallelism

Mathieu Léonardon



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom



Sauf pour les images dont la source est identifiée.

1 Architecture d'un processeur

2 Parallélisme d'instruction

- Performance
- Pipeline
- Techniques avancées

3 Hiérarchie mémoire

1 Architecture d'un processeur

2 Parallélisme d'instruction

- Performance
- Pipeline
- Techniques avancées

3 Hiérarchie mémoire

De quoi est fait un CPU (Central Processing Unit) ?

- Un CPU est constitué de millions de transistors
- Traite des données binaires
- Mémorisation
 - Registers
 - SRAM / DRAM
 - ...
- Calculs
 - Arithmétique
 - Logique
 - Contrôle

De quoi est fait un CPU (Central Processing Unit) ?

- Un CPU est constitué de millions de transistors
- Traite des données binaires
- Mémorisation
 - Registers
 - SRAM / DRAM
 - ...
- Calculs
 - Arithmétique
 - Logique
 - Contrôle

De quoi est fait un CPU (Central Processing Unit) ?

- Un CPU est constitué de millions de transistors
- Traite des données binaires
- Mémorisation
 - Registers
 - SRAM / DRAM
 - ...
- Calculs
 - Arithmétique
 - Logique
 - Contrôle

De quoi est fait un CPU (Central Processing Unit) ?

- Un CPU est constitué de millions de transistors
- Traite des données binaires
- Mémorisation
 - Registers
 - SRAM / DRAM
 - ...
- Calculs
 - Arithmétique
 - Logique
 - Contrôle

Qu'est-ce qu'un CPU (Central Processing Unit) ?

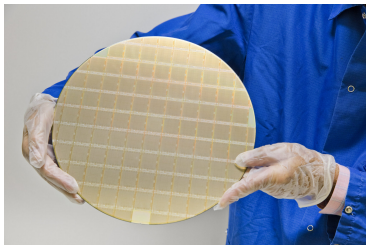


Figure: Un wafer

Qu'est-ce qu'un CPU (Central Processing Unit) ?



Figure: Un "package" CPU

Architecture d'un CPU ?

■ Architecture du Jeu d'Instructions Instruction Set Architecture - ISA

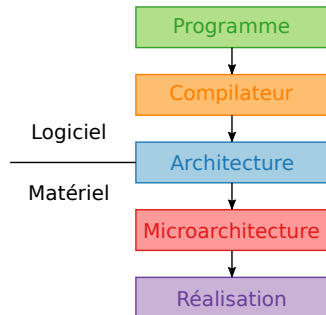
- Décrit le fonctionnement du processeur pour le compilateur
- Opérations, types de données, registres visibles, adressage

■ Microarchitecture

- PdV Concepteur du processeur
- Circuit logique qui implémente le jeu d'instructions
- Registres physiques, unités fonctionnelles, pipeline

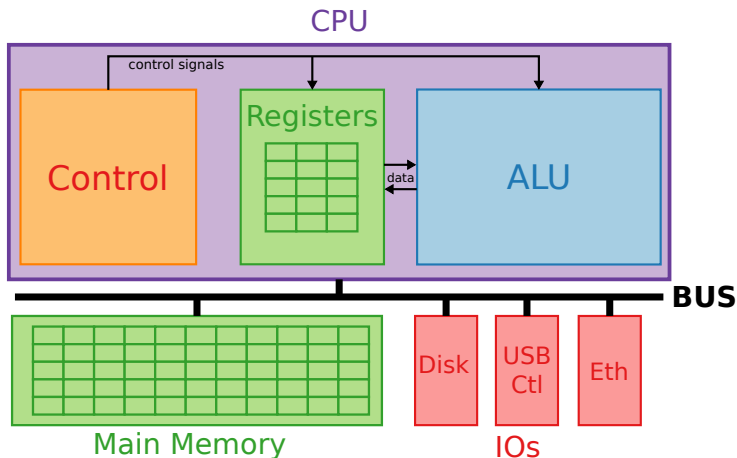
■ Réalisation

- PdV concepteur du circuit
- Réalisation physique
- Transistors, technologie utilisée, connexions



Slide inspirée du cours INF8505 - Processeurs embarqués configurables - donné par Pierre Langlois @ PolyMtl - Creative Commons

Structure d'un ordinateur



RISC-V

- Ouvert
- Récent
- RISC : Reduced Instruction Set Computer
- Destiné a priori à l'embarqué



Exemple d'Architecture : RISC-V

- Documentation complète sur Moodle
- Nous considérons la version RV32I (p.13)
- Format des données : 32 bits (instructions et données)
- 32 registres 32-bits (x0-x31) plus 1 : program counter (p.14)
- 40 instructions (p.130)
- 6 types d'instructions

31	27	26	25	24	20	19	15	14	12	11	7	6	0	
funct7				rs2		rs1		funct3		rd		opcode		R-type
imm[11:0]						rs1		funct3		rd		opcode		I-type
imm[11:5]				rs2		rs1		funct3		imm[4:0]		opcode		S-type
imm[12 10:5]				rs2		rs1		funct3		imm[4:1 11]		opcode		B-type
imm[31:12]										rd		opcode		U-type
imm[20 10:1 11 19:12]										rd		opcode		J-type

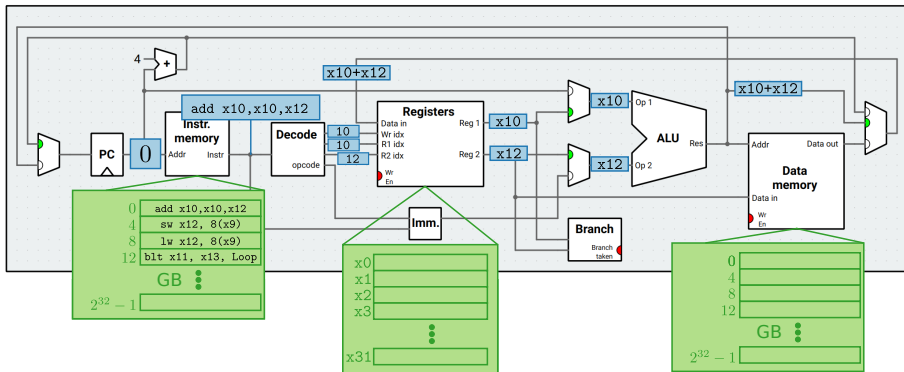
■ Exemple : instruction ADD - R-type

00000000	rs2	rs1	000	rd	0110011	ADD
----------	-----	-----	-----	----	---------	-----

- Code machine illisible pour humain
- Recours à des mnémoniques (utilisées dans le langage assembleur)
- Ex: `add x10, x10, x12`
- Cette instruction ajoute la valeur de `x10` à celle de `x12` et stocke le résultat dans `x10`
- Ce qui correspond à l'instruction 32 bits suivante:

00000000	rs2	rs1	000	rd	0110011	ADD
	01100 x12	01010 x10		01010 x10		

La microarchitecture RISC-V



Exemple : `add x10, x10, x12`

■ Register-Register operations (R-type)

- add, slt, sltu, and, or, xor, sll, srl, sub, sra
- Ex: add x10, x10, x12

■ Register-Immediate instructions (I-type)

- addi, slti, sltiu, andi, ori, xori, slli, slri
- Exemple : addi x2, x1, 40 → Ajoute 40 à la valeur de x1 et stocke dans x2.

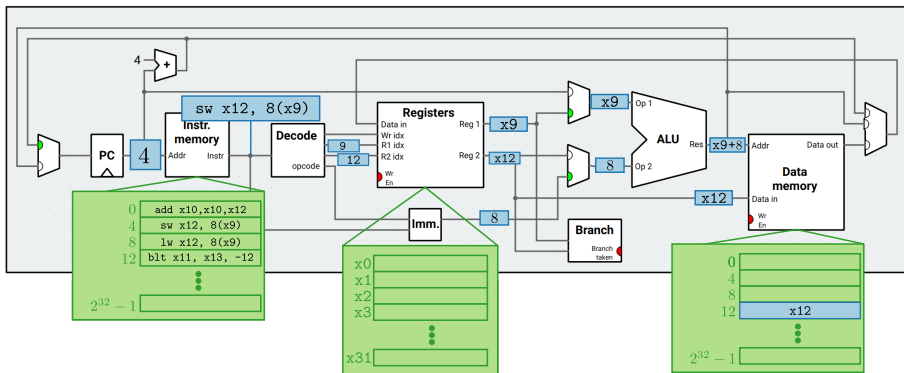
■ Load Instructions

- `lw, lh, lhu, lb, lbu`
- Ex: `lw x12, 8(x9)` → Charge depuis la donnée en mémoire à l'adresse $8 + x9$ dans le registre `x12`

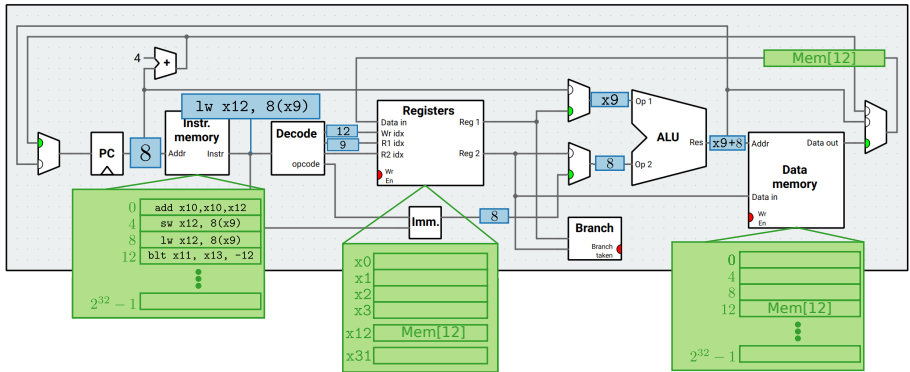
■ Store instruction

- `sw`
- Exemple : `sw x12, 8(x9)` → Sauve la valeur du registre `x12` à l'adresse $8 + x9$

Exemple : sw x12, 8(x9)



Exemple : lw x12, 8(x9)



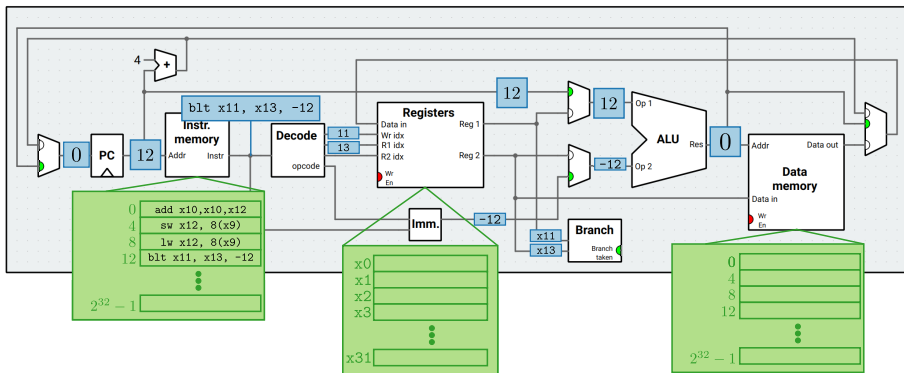
■ Unconditional Jumps (J-type)

- `jal, jalr`
- Ex: `jal Loop` → Saute à l'adresse `Loop` (`Loop` est un label pour une adresse mémoire)

■ Conditional Branches (B-type)

- `blt, beq`
- Exemple : `blt x11, x13, Loop` → Saute à l'adresse `Loop` si `x11` plus petit que `x13`

Exemple: blt x11, x13, -12



1 Architecture d'un processeur

2 Parallélisme d'instruction

- Performance
- Pipeline
- Techniques avancées

3 Hiérarchie mémoire

Performance d'un processeur

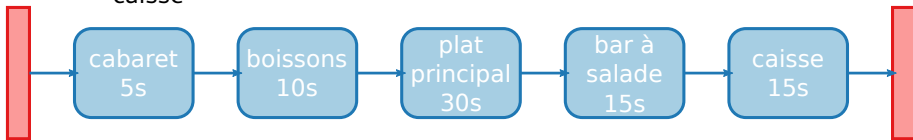
- Pas simple à définir
- Peut dépendre de l'application
 - Usage général
 - Domaine scientifique (e.g Météorologie)
 - Serveur web
 - Calculs Financiers
- Pour les systèmes embarqués, il faut aussi tenir compte de la puissance et de la surface

- Quelques définitions : latence vs débit
- Analyse et équation de la performance d'un CPU
- Principes de design pour améliorer la performance

Latence et débit

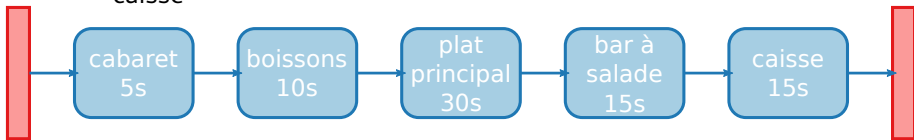
- Soit une chaîne de cafétéria avec 5 stations
- Supposons
 - une seul client par ligne à la fois
 - chaque client passe par chaque stations
 - Un client ne peu pas commencer la ligne tant que le précédent n'a pas fini à la caisse

- Période = ; fréquence =
- Latence : 1 cycle = secondes pour servir un client
- Débit :



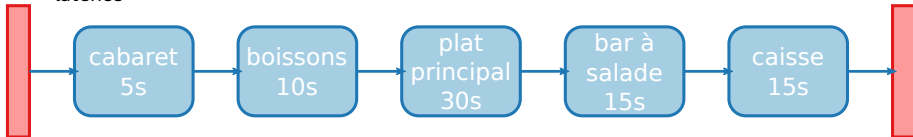
Latence et débit

- Soit une chaîne de cafétéria avec 5 stations
- Supposons
 - une seul client par ligne à la fois
 - chaque client passe par chaque stations
 - Un client ne peu pas commencer la ligne tant que le précédent n'a pas fini à la caisse
- Période = 75s; fréquence = 13.3 mHz
- Latence : 1 cycle = 75 secondes pour servir un client
- Débit : 1 client / 75s = 48 clients par heure



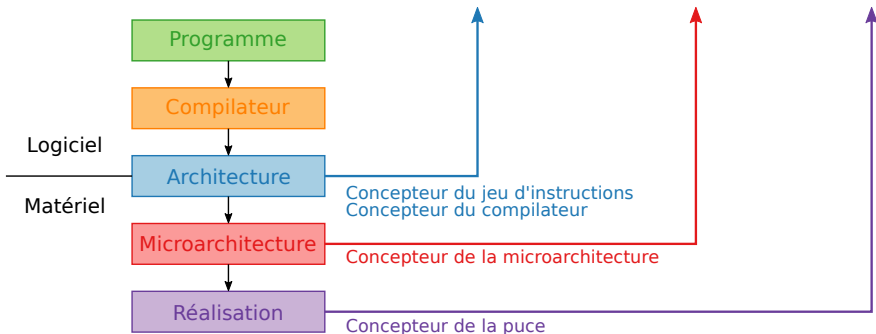
Latence et débit

- **La Latence de calcul est le temps nécessaire pour produire un résultat**
- C'est le temps entre le moment où une donnée est disponible et le moment où le résultat qui dépend de cette donnée est disponible à son tour
- Courte latence = meilleure performance
- La latence est souvent exprimée en nombre de cycles d'horloge
- Une fréquence d'horloge élevée correspond à une latence plus courte
- Le parallélisme n'a pas d'influence sur la latence
- **Le débit d'information est le nombre de résultats produits par unité de temps**
- On suppose qu'une quantité suffisante de données est disponible à l'entrée du système pour le garder toujours actif
- Grand débit = meilleure performance
- Le débit est exprimé en nombre de résultats par secondes
- Une fréquence d'horloge élevée correspond à un débit plus grand
- Le parallélisme augmente le débit



Equation de la performance

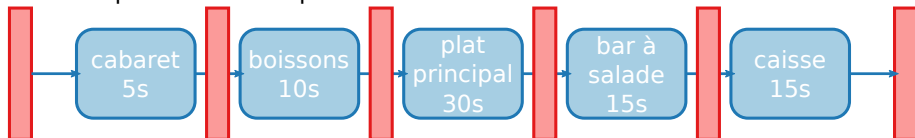
$$\frac{1}{P} = \frac{\text{Temps}}{\text{Programme}} = \frac{\#Instructions}{\text{Programme}} \cdot \frac{\#Cycles}{\text{Instruction}} \cdot \frac{\text{Temps}}{\text{Cycle}}$$



Pipeline

- Considérons une cafétéria avec 5 stations
- Supposons
 - Il peut y avoir un client à chaque station, mais pas plus d'un client par station

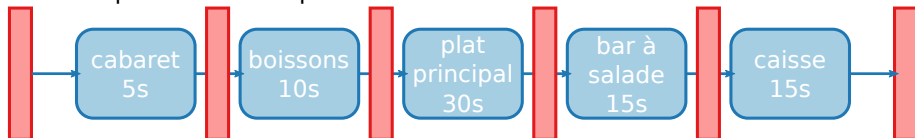
- Période =
- Fréquence =
- Latence =
- Débit =



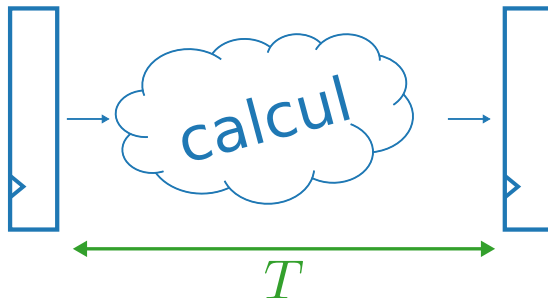
Pipeline

- Considérons une cafétéria avec 5 stations
- Supposons
 - Il peut y avoir un client à chaque station, mais pas plus d'un client par station

- Période = 30s
- Fréquence = 33.3 mHz
- Latence = 5 cycles = 150s
- Débit = 120 cli/h

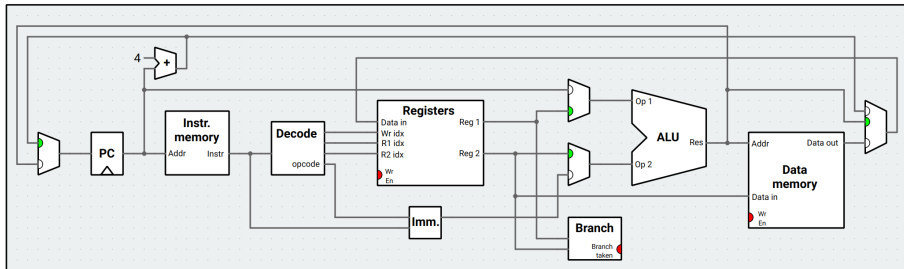


Circuit synchrone



- Chemin critique : chemin le plus long (T_0)
- Fréquence du CPU : $\frac{1}{T_0}$

Chemin critique



- Chemin critique : chemin le plus long (T_0)
- Fréquence du CPU : $\frac{1}{T_0}$



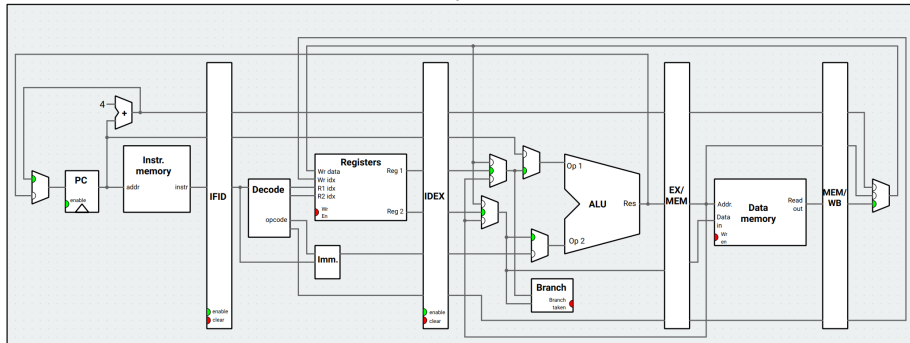
- Fetch: Récupérer une instruction en mémoire
- Decode: Lire et décoder l'instruction, la transformer en signaux de contrôle
- Execute: Réaliser les opérations, calculs
- Memory: réaliser les opérations mémoire (sauver, charger)
- Write Back: Ecrire le résultat dans un registre

Pipeline RISC-V



	0	1	2	3	4	5	6	7	8	9	10	11	12
<code>auipc x10 0x10000</code>	IF	ID	EX	MEM	WB								
<code>lw x10 0(x10)</code>		IF	ID	EX	MEM	WB							
<code>auipc x11 0x10000</code>			IF	ID	EX	MEM	WB						
<code>lw x11 -4(x11)</code>				IF	ID	EX	MEM	WB					
<code>auipc x12 0x10000</code>					IF	ID	EX	MEM	WB				
<code>lw x12 -8(x12)</code>						IF	ID	EX	MEM	WB			
<code>auipc x13 0x10000</code>							IF	ID	EX	MEM	WB		
<code>lw x13 -12(x13)</code>								IF	ID	EX	MEM	WB	
<code>jal x1 0x84 <complexMul></code>									IF	ID	EX	MEM	WB

Pipeline RISC-V



Pipeline idéal d'un processeur



	0	1	2	3	4	5	6	7	8	9	10	11	12
<code>auipc x10 0x10000</code>	IF	ID	EX	MEM	WB								
<code>lw x10 0(x10)</code>		IF	ID	EX	MEM	WB							
<code>auipc x11 0x10000</code>			IF	ID	EX	MEM	WB						
<code>lw x11 -4(x11)</code>				IF	ID	EX	MEM	WB					
<code>auipc x12 0x10000</code>					IF	ID	EX	MEM	WB				
<code>lw x12 -8(x12)</code>						IF	ID	EX	MEM	WB			
<code>auipc x13 0x10000</code>							IF	ID	EX	MEM	WB		
<code>lw x13 -12(x13)</code>								IF	ID	EX	MEM	WB	
<code>jal x1 0x84 <complexMul></code>									IF	ID	EX	MEM	WB

Pipeline : problèmes (hazards)

■ Conflits structurels

- dus à manque de ressources (e.g. une seule mémoire pour les données et les instructions, ou bien utilisation de l'ALU pour les opérations arithmétiques et pour le calcul d'une adresse)

■ Conflits de données

- si deux instructions successives produisent des résultats interdépendants

■ Conflits de contrôle

- causés par des branchements

Pipeline : conflit de données



- **ADD** x1, x4, x5
- **AND** x8, x1, x3
- **OR** x9, x6, x1
- **SUB** x1, x4, x5

Pipeline : conflit de données



- **ADD** x1, x4, x5
- **AND** x8, x1, x3
- **OR** x9, x6, x1
- **SUB** x1, x4, x5

Pipeline : conflit de données



- **ADD** x1, x4, x5
- **AND** x8, x1, x3
- **OR** x9, x6, x1
- **SUB** x1, x4, x5

Pipeline : conflit de données



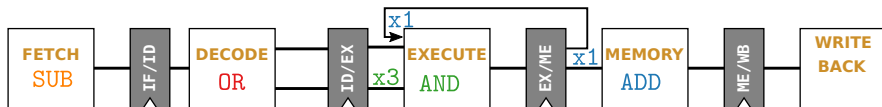
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5

Pipeline : conflit de données



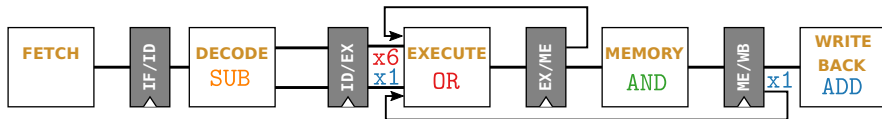
- ADD x1, x4, x5
- nop
- nop
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5

Pipeline : conflit de données



- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5

Pipeline : conflit de données



- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



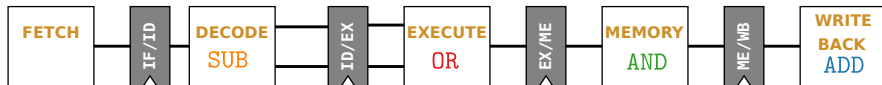
- **BEQ** x9, x8, L1
- **ADD** x1, x4, x5
- **AND** x8, x1, x3
- **OR** x9, x6, x1
- **SUB** x1, x4, x5
- ...
- L1: **XOR** x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



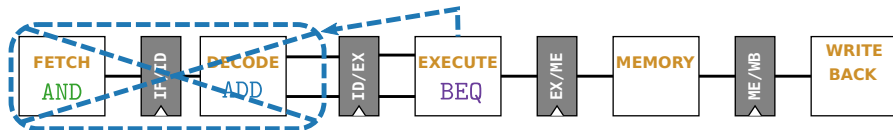
- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle



- BEQ x9, x8, L1
- ADD x1, x4, x5
- AND x8, x1, x3
- OR x9, x6, x1
- SUB x1, x4, x5
- ...
- L1: XOR x4, x6, x1

Pipeline : conflit de contrôle - Multiples solutions - exécution spéculative

- **Option 1: Bloquer le pipeline quand un branchement conditionnel est détecté.**
- Option 2: Supposer que le branchement ne sera pas fait et continuer à charger le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 3: Supposer que le branchement sera fait et charger cette instruction dans le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 4: Doubler le pipeline et exécuter les deux instructions, puis choisir quand on a déterminé si le branchement se fait ou non.
- Option 5: Prédire dynamiquement si le branchement sera fait ou non, selon le code d'opération (e.g. BEQ: prédire oui; BPL: prédire non).
- Option 6: Prédire dynamiquement si le branchement sera fait ou non, selon les caractéristiques du programme exécuté à ce moment.
- Option 7: Utiliser un tampon de boucle qui contient toutes les instructions d'une boucle. Réduit les pénalités de branchement parce que les instructions sont disponibles.

Pipeline : conflit de contrôle - Multiples solutions - exécution spéculative

- Option 1: Bloquer le pipeline quand un branchement conditionnel est détecté.
- Option 2: Supposer que le branchement ne sera pas fait et continuer à charger le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 3: Supposer que le branchement sera fait et charger cette instruction dans le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 4: Doubler le pipeline et exécuter les deux instructions, puis choisir quand on a déterminé si le branchement se fait ou non.
- Option 5: Prédire dynamiquement si le branchement sera fait ou non, selon le code d'opération (e.g. BEQ: prédire oui; BPL: prédire non).
- Option 6: Prédire dynamiquement si le branchement sera fait ou non, selon les caractéristiques du programme exécuté à ce moment.
- Option 7: Utiliser un tampon de boucle qui contient toutes les instructions d'une boucle. Réduit les pénalités de branchement parce que les instructions sont disponibles.

Pipeline : conflit de contrôle - Multiples solutions - exécution spéculative

- Option 1: Bloquer le pipeline quand un branchement conditionnel est détecté.
- Option 2: Supposer que le branchement ne sera pas fait et continuer à charger le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 3: Supposer que le branchement sera fait et charger cette instruction dans le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 4: Doubler le pipeline et exécuter les deux instructions, puis choisir quand on a déterminé si le branchement se fait ou non.
- Option 5: Prédire dynamiquement si le branchement sera fait ou non, selon le code d'opération (e.g. BEQ: prédire oui; BPL: prédire non).
- Option 6: Prédire dynamiquement si le branchement sera fait ou non, selon les caractéristiques du programme exécuté à ce moment.
- Option 7: Utiliser un tampon de boucle qui contient toutes les instructions d'une boucle. Réduit les pénalités de branchement parce que les instructions sont disponibles.

Pipeline : conflit de contrôle - Multiples solutions - exécution spéculative

- Option 1: Bloquer le pipeline quand un branchement conditionnel est détecté.
- Option 2: Supposer que le branchement ne sera pas fait et continuer à charger le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 3: Supposer que le branchement sera fait et charger cette instruction dans le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 4: Doubler le pipeline et exécuter les deux instructions, puis choisir quand on a déterminé si le branchement se fait ou non.
- Option 5: Prédire dynamiquement si le branchement sera fait ou non, selon le code d'opération (e.g. BEQ: prédire oui; BPL: prédire non).
- Option 6: Prédire dynamiquement si le branchement sera fait ou non, selon les caractéristiques du programme exécuté à ce moment.
- Option 7: Utiliser un tampon de boucle qui contient toutes les instructions d'une boucle. Réduit les pénalités de branchement parce que les instructions sont disponibles.

Pipeline : conflit de contrôle - Multiples solutions - exécution spéculative

- Option 1: Bloquer le pipeline quand un branchement conditionnel est détecté.
- Option 2: Supposer que le branchement ne sera pas fait et continuer à charger le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 3: Supposer que le branchement sera fait et charger cette instruction dans le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 4: Doubler le pipeline et exécuter les deux instructions, puis choisir quand on a déterminé si le branchement se fait ou non.
- Option 5: Prédire dynamiquement si le branchement sera fait ou non, selon le code d'opération (e.g. BEQ: prédire oui; BPL: prédire non).
- Option 6: Prédire dynamiquement si le branchement sera fait ou non, selon les caractéristiques du programme exécuté à ce moment.
- Option 7: Utiliser un tampon de boucle qui contient toutes les instructions d'une boucle. Réduit les pénalités de branchement parce que les instructions sont disponibles.

Pipeline : conflit de contrôle - Multiples solutions - exécution spéculative

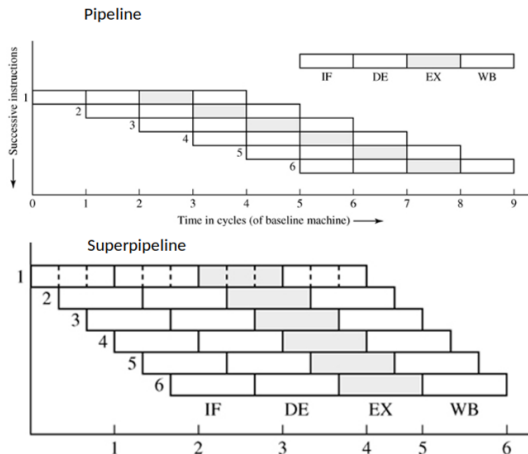
- Option 1: Bloquer le pipeline quand un branchement conditionnel est détecté.
- Option 2: Supposer que le branchement ne sera pas fait et continuer à charger le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 3: Supposer que le branchement sera fait et charger cette instruction dans le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 4: Doubler le pipeline et exécuter les deux instructions, puis choisir quand on a déterminé si le branchement se fait ou non.
- Option 5: Prédire dynamiquement si le branchement sera fait ou non, selon le code d'opération (e.g. BEQ: prédire oui; BPL: prédire non).
- Option 6: Prédire dynamiquement si le branchement sera fait ou non, selon les caractéristiques du programme exécuté à ce moment.
- Option 7: Utiliser un tampon de boucle qui contient toutes les instructions d'une boucle. Réduit les pénalités de branchement parce que les instructions sont disponibles.

Pipeline : conflit de contrôle - Multiples solutions - exécution spéculative

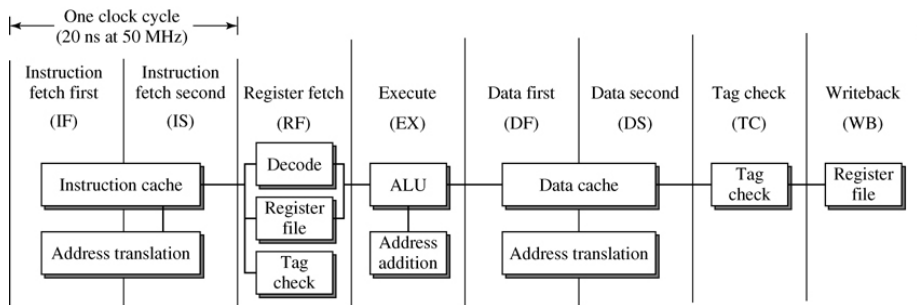
- Option 1: Bloquer le pipeline quand un branchement conditionnel est détecté.
- Option 2: Supposer que le branchement ne sera pas fait et continuer à charger le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 3: Supposer que le branchement sera fait et charger cette instruction dans le pipeline. Il faut flush les instructions si le branchement est fait.
- Option 4: Doubler le pipeline et exécuter les deux instructions, puis choisir quand on a déterminé si le branchement se fait ou non.
- Option 5: Prédire dynamiquement si le branchement sera fait ou non, selon le code d'opération (e.g. BEQ: prédire oui; BPL: prédire non).
- Option 6: Prédire dynamiquement si le branchement sera fait ou non, selon les caractéristiques du programme exécuté à ce moment.
- Option 7: Utiliser un tampon de boucle qui contient toutes les instructions d'une boucle. Réduit les pénalités de branchement parce que les instructions sont disponibles.

Superpipeline

- Un processeur superpipeliné contient un nombre élevé d'étages de pipeline. Certaines des étapes 'traditionnelles' du pipeline sont décomposées en plusieurs étapes.
- C'est une façon de mieux répartir la latence des différentes étapes du cycle des instructions sur plusieurs étages.

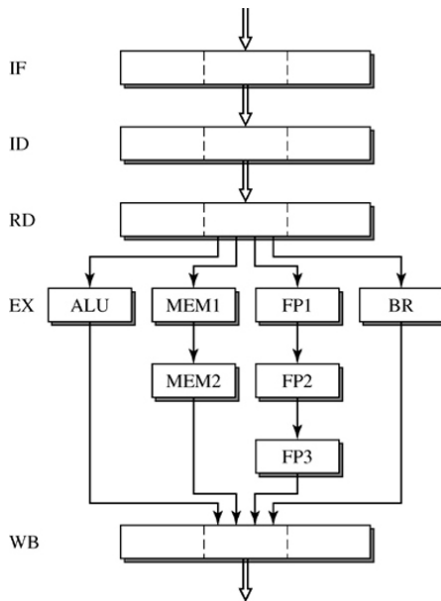


Exemple : MIPS R4000 (1991)



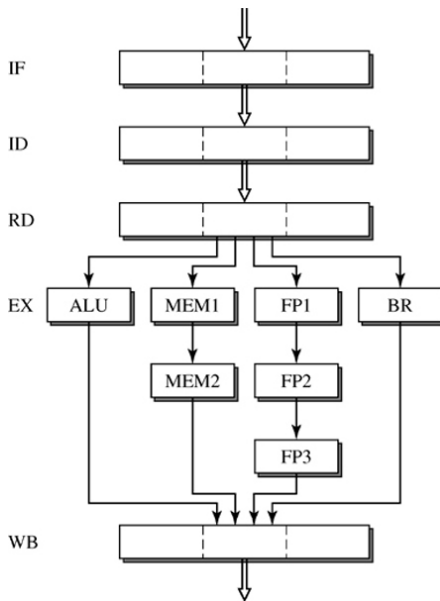
Superscalaire

- Un processeur scalaire peut lancer une seule instruction à la fois (mais plusieurs peuvent être en cours d'exécution grâce à un pipeline).
- Un processeur superscalaire peut:
 - lancer plusieurs instructions simultanément,
 - exécuter plusieurs instructions de façon concurrente et indépendante dans plusieurs pipelines,
 - exécuter des instructions dans un ordre différent de l'ordre prescrit par le programme: *out-of-order*



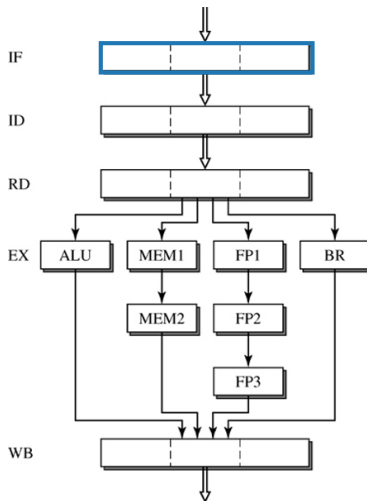
Conditions pour un bon fonctionnement superscalaire

- Le parallélisme matériel est une mesure de l'habileté du processeur à exécuter des instructions indépendantes.
- Le parallélisme matériel dépend:
 - du nombre d'instructions qui peuvent être chargées simultanément;
 - du nombre de pipelines parallèles;
 - des mécanismes présents pour détecter le parallélisme dans les opérations.



VLIW : Very Long Instruction Word

- Désavantages de l'approche superscalaire:
 - beaucoup d'efforts à ordonner les instructions
 - beaucoup d'efforts pour assigner une instruction à une unité fonctionnelle
- Une solution: laisser le compilateur régler ces problèmes en combinant plusieurs instructions dans un seul (très loooooooooonng) mot (48, 64 ++ bits).
- On ne fait que déplacer le problème, mais le compilateur a beaucoup plus de temps pour effectuer une organisation du programme 'optimale'.



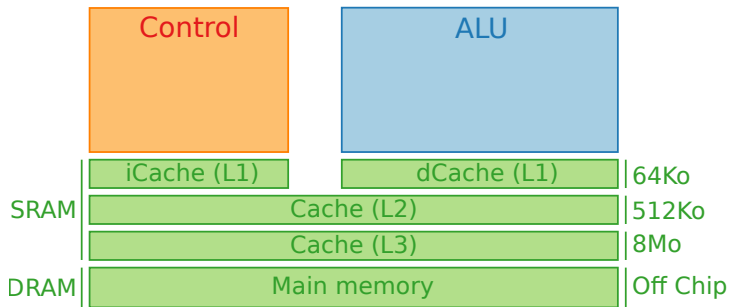
1 Architecture d'un processeur

2 Parallélisme d'instruction

- Performance
- Pipeline
- Techniques avancées

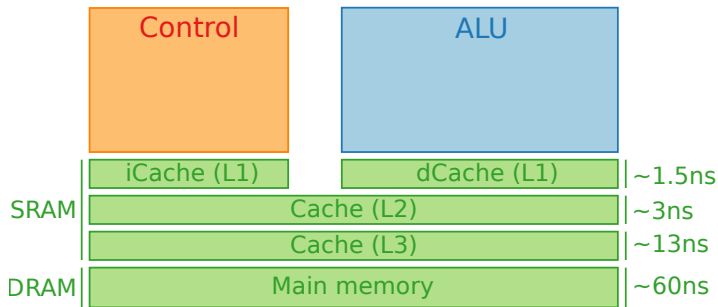
3 Hiérarchie mémoire

Hiérarchie de la mémoire



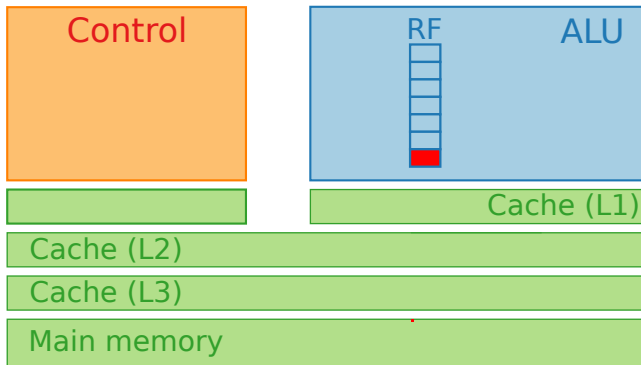
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

Hiérarchie de la mémoire



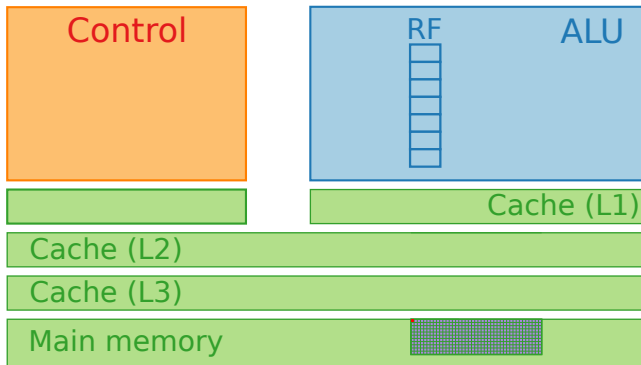
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

Hiérarchie de la mémoire



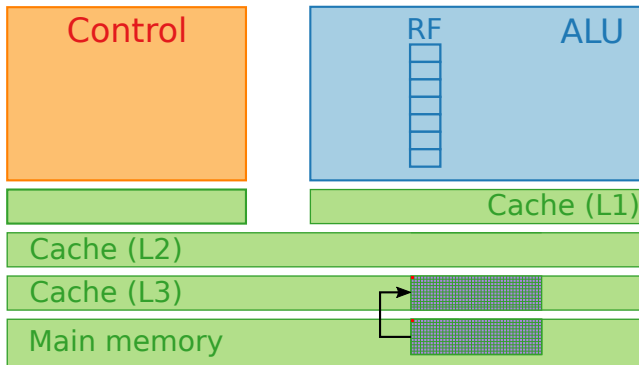
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
 - Cache Hit
 - Cache Miss

Hiérarchie de la mémoire



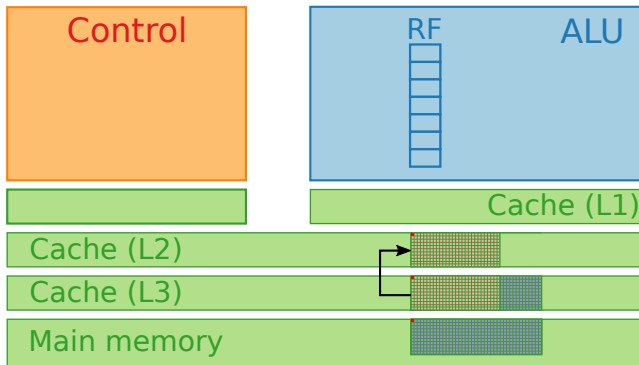
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

Hiérarchie de la mémoire



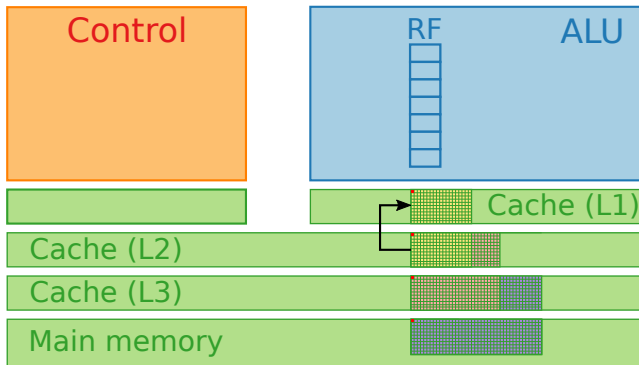
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

Hiérarchie de la mémoire



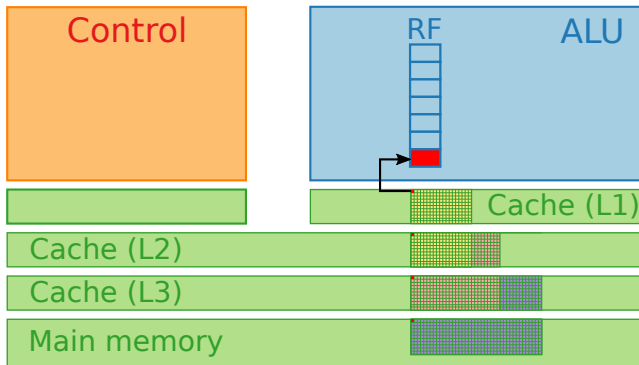
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

Hiérarchie de la mémoire



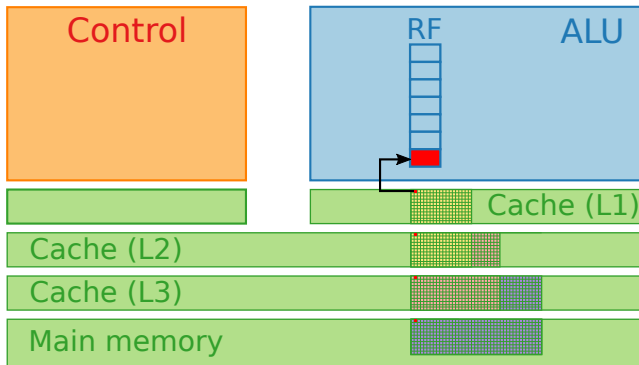
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
 - Cache Hit
 - Cache Miss

Hiérarchie de la mémoire



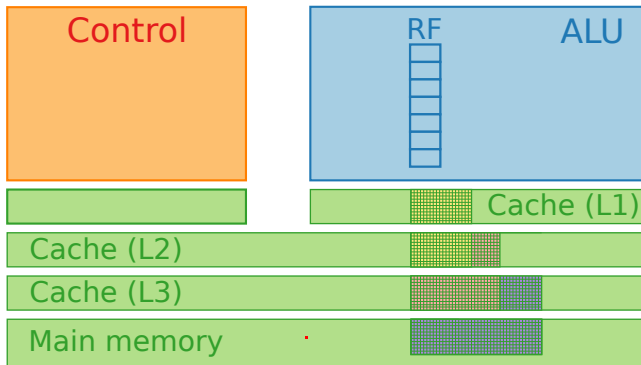
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

Hiérarchie de la mémoire



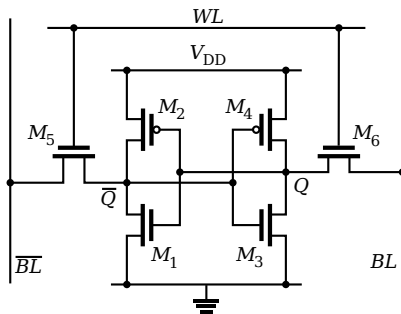
- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

Hiérarchie de la mémoire

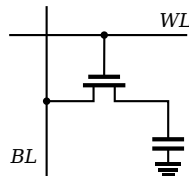


- Hiérarchie de la cache
- SRAM vs DRAM
- Premier accès
- Cache Hit
- Cache Miss

SRAM vs DRAM



SRAM



DRAM

- SRAM 6T (typically) vs DRAM 1T
- SRAM plus chère
- DRAM plus dense
- DRAM nécessite un rafraîchissement
- SRAM est plus rapide